

BRDS-PEC: Behavioral Ransomware Detection System

Pre-Encryption Containment Prototype — Technical Architecture & Codebase Review

1. Executive Overview & Core Concept

The **Behavioral Ransomware Detection System with Pre-Encryption Containment (BRDS-PEC)** is a specialized security research prototype designed to detect and halt zero-day ransomware attacks before widespread file encryption takes place. Rather than relying on static file hashes or known signatures—which are easily bypassed by code obfuscation, re-packing, or minor binary mutations—BRDS-PEC continuously monitors Windows System Monitor (Sysmon) event telemetry. It evaluates behavioral sequences (process creation, file modifications, registry changes, network socket connections, and shadow copy deletions) to answer the critical question: *'Is this process behaving like ransomware before encryption completes?'*

2. End-to-End Execution Flow

```
Windows Sysmon / Exported EVT_X or XML Logs
↓ Parse and Normalize Event Data (evtx_reader.py)
↓ Filter Relevant Security Events (event_filter.py)
↓ Aggregate per Process in 5-Second Windows (temporal_aggregator.py)
↓ Produce 17 Numeric Behavioral Features (vectorizer.py)
↓ Baseline Model & PyTorch LSTM Produce Risk Scores (ml_engine/)
↓ Alert Stored, Signed with HMAC-SHA256, and Displayed in Dashboard
↓ Containment Daemon Acts Only When Explicitly Armed (-Armed Switch)
```

3. Core Root Files

- **README.md**: Describes high-level architecture, goals, and setup. Serves as design intent.
- **requirements.txt**: Outlines core dependencies: pandas, numpy, python-evtx, scikit-learn, joblib, torch, shap, Flask, Flask-SQLAlchemy, Flask-Cors, python-dotenv, requests.
- **.env.example**: Production template defining key safe configurations: BRDS_DRY_RUN=1 (safe dry-run mode), BRDS_API_KEY (ingestion auth), BRDS_ALERT_HMAC_KEY (alert signature key), and BRDS_CORS_ORIGINS (dashboard domain allowlist).

4. Data Folder & Datasets Breakdown (data/)

The `data/` folder is structured into three sub-directories: `data/datasets/` (immutable research data), `data/processed/` (generated feature windows & alerts), and `data/models/` (trained models & scalers).

Dataset / Artifact	Key Contents	Role & Limitations in Live Pipeline
CSU Ransomware data/datasets/csu_ransomware/	352k rows, 20 behavior columns (File_created, Process_Create, entropy, etc.).	Useful for offline anomaly baselines. Not directly plugged into live Sysmon pipeline due to schema mismatch.

RansomSet data/datasets/ransoms et/	API frequency CSVs (NtCreateFile, NtWriteFile, registry APIs) for 7 families.	Auxiliary multiclass benchmark for family comparison. Contains sandbox API calls, not raw Sysmon logs.
MLRan data/datasets/mlran/	Sample metadata, Cuckoo parsers, goodwill descriptions, feature-selected CSVs.	Informs static/dynamic hybrid classification research. Selected features reflect sandbox Cuckoo reports.
OTRF Security data/datasets/otrf_sec urity_datasets/	Defensive-evasion scenarios (disabling event logs, command line, PowerShell).	Tests preparatory behavior (defense impairment). Supporting scenarios, not a complete ransomware dataset.
Splunk ATT&CK; Data data/datasets/splunk_a ttack_data/	Raw Sysmon logs for T1059.001, T1105, T1486 (encryption), T1490 (shadow copy deletion).	Primary source for live pipeline. Raw timestamped telemetry parsed directly into behavioral windows.

5. Telemetry Engineering (pipeline/)

- **evtx_reader.py:** Reads native .evtx files via python-evtx or Splunk XML. Extracts Event ID, process GUID/PID, image, parent image, command line, target file, registry target, and destination IP/port.
- **event_filter.py:** Filters Sysmon Event IDs 1 (process creation), 3 (network), 7 (DLL load), 11 (file create), 12/13 (registry), 23/26 (file delete). Rejects events lacking timestamps or process GUIDs.
- **flatten_encode.py:** Path normalization, extension extraction, and suspicious path flag (e.g., Temp, AppData, Downloads, Public).
- **temporal_aggregator.py:** Core windowing engine. Groups by computer + process_key + 5-second UTC window. Computes 17 features including event counts, unique file/extension/IP counts, and per-Event-ID metrics.
- **vectorizer.py:** Strips metadata columns, produces numeric feature matrices, and fills missing values with zero.
- **watchdog.py:** Tracks event arrival intervals. If telemetry stalls for >30 seconds, raises a SENSOR_SILENCED security alert.

6. Workflow Entry Points (scripts/)

- **run_pipeline.py:** Builds windowed CSV datasets from raw Sysmon log folders. Assigns label=1 to attack windows and label=0 to benign windows.
- **train_baseline.py:** Trains baseline models using **source-level splits** (preventing scenario leakage). Trains Isolation Forest (anomaly screening) and Logistic Regression (class_weight='balanced', max_iter=2000). Reports Precision, Recall, F1, ROC-AUC, FPR, and lead times.
- **score_windows.py:** Loads baseline artifacts, evaluates sequence windows against the 0.85 risk threshold, and writes HMAC-signed alert containers.
- **prepare_live_data.py:** Demonstration generator mixing Splunk attack windows with 2,000 synthetic benign windows, training models, creating alerts, and seeding SQLite.
- **extract_goodware.py:** Utility extracting benign telemetry from CSU, MLRan, and RansomSet.

7. Detection & Explanation Engine (ml_engine/)

- **risk_engine.py:** Scores incoming vectors, computes supervised probability and Isolation Forest anomaly scores, and determines alert triggers.
- **lstm/dataset.py:** Formats 30-step sequence tensors (150 seconds of history) per process, enabling sequence distinction between isolated commands and full ransomware execution chains.
- **lstm/model.py:** Bidirectional 2-layer LSTM featuring hidden projection, concatenated Mean + Max pooling across 30 timesteps, and Sigmoid risk output.
- **lstm/infer.py:** Real-time inference wrapper enforcing weights_only=True, verifying SHA-256 model checksums, restoring scalars, and applying feature mean-padding on short sequences.
- **xai/shap_explainer.py:** Provides SHAP and PyTorch autograd integrated gradient attributions ($|\nabla_x y * x|$) to explain feature drivers to SOC analysts.

8. Backend API, Persistence & SOC Dashboard

- **Backend (backend/):** Flask application factory (app.py), SQLite ORM models (brds.db), constant-time @require_api_key authentication (auth.py), and REST routes (telemetry_routes, incident_routes, xai_routes).
- **Frontend (frontend/):** Dark-mode SOC dashboard (index.html, dashboard.css) with real-time Chart.js rolling risk curves (risk_timeline.js), live Sysmon event streams (telemetry_stream.js), active incident logs (incident_log.js), and XAI attribution modals (xai_modal.js).

9. Containment Response Mechanism (containment/)

- **alert_integrity.py:** Generates HMAC-SHA256 digests for alerts and issues single-use .arm_token files.
- **trigger_daemon.py:** Polls signed alerts every 1.5s, verifies signatures, checks 0.85 risk threshold, validates arm tokens, and invokes PowerShell scripts.
- **ContainHost.ps1:** Disables active NICs, clears ARP/DNS caches, and applies Windows Firewall block rules. Operates in dry-run mode unless executed with -Armed.
- **kill_process_tree.ps1:** Recursively terminates ransomware process trees while checking the \$PROTECTED_PROCESSES denylist (lsass, csrss, svchost, services, etc.) to prevent BSOD crashes.

10. Important Guidelines & Pre-Submission Review Checklist

- **1. Accuracy & FPR Evaluation:** Do not claim operational real-world accuracy or false-positive rates unless evaluated against genuine, clean endpoint Sysmon logs and held-out ransomware scenarios.
- **2. Synthetic Benign Data Transparency:** prepare_live_data.py uses synthetic benign windows for demonstration purposes. Clearly state that synthetic data is for integration testing, not operational benchmark claims.
- **3. Model Report Requirements:** Submissions must specify exact data sources, ransomware families, benign collection methods, source-level split strategies, confusion matrices, and encryption-start lead times.
- **4. Frontend Simulation Fallbacks:** Disclose when demo UI components fall back to simulated data when live backend APIs are disconnected.
- **5. Disconnected VM Safety Rules:** Never test active containment on daily-use host machines. Execute exclusively within isolated virtual machines with snapshots, disabled network bridging, and no shared folders.
- **6. HMAC Secret Security:** Never deploy default HMAC secret keys in production. Set a strong, unique BRDS_ALERT_HMAC_KEY in environment settings.

- **7. Resolving Inconsistencies:** Note that ContainHost.ps1 and kill_process_tree.ps1 rely on the -Armed parameter switch; verify arm token validation logic prior to live deployment.
- **8. Recommended Submission Package:** Include concise architecture diagrams, dataset governance tables, clean setup commands, virtual environment test reports, dashboard screenshots, dry-run demo recordings, and safety documentation.